Speculative Decoding

Why AI Writes a Secret First Draft

A small draft model proposes several tokens. The main model scores the proposed block together in one verification pass. The algorithm then checks the candidates in order and keeps the accepted prefix.

01 From scores to probabilities

At each position, both models turn their scores (logits) into probabilities using softmax. If temperature T is used, divide each logit by T before applying softmax.

$$P(x) = \exp(z_x / T) / \text{sum}_y \exp(z_y / T)$$

Here x is one candidate token and y ranges over the vocabulary. Let q(x) be its draft probability and p(x) its main-model probability. Compare them for the same token and the same preceding context.

02 The midnight check

Radio prompt: “Meet me at the…” Draft: “bridge at midnight.” Suppose the checks accept “bridge” and “at”. Now both models score the next token after “Meet me at the bridge at”. The words stand in for tokens in this simplified example.

Acceptance chance = min(1, p(x) / q(x))
For “midnight”: min(1, 0.30 / 0.50) = **0.60**

Draw a uniform random number u between 0 and 1. If u is below 0.60, accept “midnight”. In the Reel’s run, u could be 0.74, so “midnight” is rejected. Its rejection chance is 40% after it has been proposed.

After the check

03 Where “dawn” comes from

At the first rejection, keep the earlier accepted tokens and discard the remaining draft. Sample a replacement from the main model’s leftover probability mass after subtracting the draft probabilities and clipping negative values to zero.

$$r(x) = \max(0, p(x) - q(x)) / \text{sum}_y \max(0, p(y) - q(y))$$

In this toy vocabulary, the leftover mass adds up to 0.20. The replacement draw can therefore land on “dawn” with probability 0.15 / 0.20 = 75%, given rejection.

Token	Draft q	Main p	Leftover	Replacement r
midnight	50%	30%	0%	0%
dawn	20%	35%	15%	75%
sunset	30%	35%	5%	25%

Why it works: each token gets accepted mass min(p, q) plus leftover mass max(0, p - q). Their sum equals p. This preserves the main model’s sampling distribution.

04 Where the speed comes from

One verification pass can commit several tokens. When the whole draft passes, the original algorithm also samples one extra token from the main model. Actual speed depends on acceptance, draft cost and verification cost. A pass count alone cannot predict wall-clock speed.

Quick check. If q(x) = 0.20 and p(x) = 0.35, what is the acceptance chance? **100%**, because the ratio exceeds 1 and the rule caps it at 1.

Source: Leviathan, Kalman & Matias (2023), Fast Inference from Transformers via Speculative Decoding, Algorithm 1.

Follow one decoding round

A token is a unit of text from a tokenizer. Here we use three words as stand-ins for three tokens so you can trace the algorithm by hand.

Committed prompt → Draft three tokens → Score the block → Check in order → Commit output

1. Draft from left to right

Starting from “Meet me at the…”, the draft model proposes “bridge”, then proposes “at” conditioned on “bridge”, then proposes “midnight” conditioned on both. It saves the probability distribution used at each step.

2. Score the same contexts together

The main model receives the prompt and proposed block. Causal attention lets it score each next-token position using only the tokens before that prediction. It can calculate those scores in one pass even though the acceptance decisions still happen in order.

3. Walk the acceptance checks

Illustrative numbers for this run. The first two probability pairs are additional teaching examples.

bridge: p = 0.48, q = 0.40, accept chance = 1.00. Draw 0.21 → keep.
at: p = 0.63, q = 0.70, accept chance = 0.90. Draw 0.42 → keep.
midnight: p = 0.30, q = 0.50, accept chance = 0.60. Draw 0.74 → reject.

4. Commit and continue

The replacement draw lands on “dawn” in this run. The committed output is “bridge at dawn”. Both models continue from that new prefix. Any draft tokens after the rejected token would be discarded, because their context included “midnight”.

If all three proposals pass, the original algorithm samples a fourth token from the main model using the full accepted prefix.

Prove it, then measure it

Why the final probabilities still match

At the “midnight” position, consider every possible draft draw. The accepted mass is 0.30 + 0.20 + 0.30 = 0.80. The total rejection probability at this position is 0.20. That differs from the 40% rejection chance conditional on proposing “midnight”.

Final midnight: 0.50 × 0.60 + 0.20 × 0 = 0.30
Final dawn: 0.20 × 1 + 0.20 × 0.75 = 0.35
Final sunset: 0.30 × 1 + 0.20 × 0.25 = 0.35

The three totals match the main model’s probabilities. Across runs, this sampling rule preserves the distribution. Individual runs can produce different outputs.

How to estimate speed

Suppose three baseline main-model steps take 30 ms each. In an illustrative round, three draft steps take 5 ms each and block verification takes 40 ms. If the round commits three tokens, the comparison is 90 ms versus 55 ms, or about 1.64× faster. If that same round commits only one token, 55 ms is slower than the 30 ms baseline for one token.

Benchmark the same prompts and decoding settings with and without speculation. Compare output tokens per second, time to first token and end-to-end latency. Log accepted draft tokens and committed tokens per round so you can explain the difference.

Check your understanding

1. If the first candidate is rejected, can a later draft candidate survive? **No.** The algorithm restarts from the accepted prefix plus the replacement.
2. Why is a 60% acceptance chance different from a 30% final probability? **60%** is conditional on the draft proposing that token. **30%** is its overall final probability.
3. Does one verification pass guarantee a speedup? **No.** Drafting and verification have costs, and a round may commit few tokens.